Hardik Tyagi

ENGINEERING CASE STUDY · MAY 2026

AI SUPPORT COPILOT

Enterprise teams need answers they can trust. This project uses retrieval-augmented generation to search internal documentation, surface relevant source passages, and generate grounded responses with citations.

Built for auditable support workflows.

NEXT.JS 14 | FASTAPI | POSTGRESQL 15 | CHROMADB | OPENAI

DOCKER

ARCHITECTURE

THE PROBLEM: NAIVE GPT DOESN'T SCALE

NAIVE GPT

- Hits token limits fast
- Gets expensive at scale
- No source traceability
- Hallucinates missing context
- Needs manual tenant filtering

RAG

- Uses top-k chunks only
- Fixed cost per query
- Claims map to sources
- Refuses without context
- Isolated per tenant

INGESTION

PDF → chunks → embeddings → store

RETRIEVAL

Query embed → top matches → context

GENERATION

Grounded prompt → answer → citation check

MULTI-TENANCY

Namespace isolation per tenant

SYSTEM ARCHITECTURE

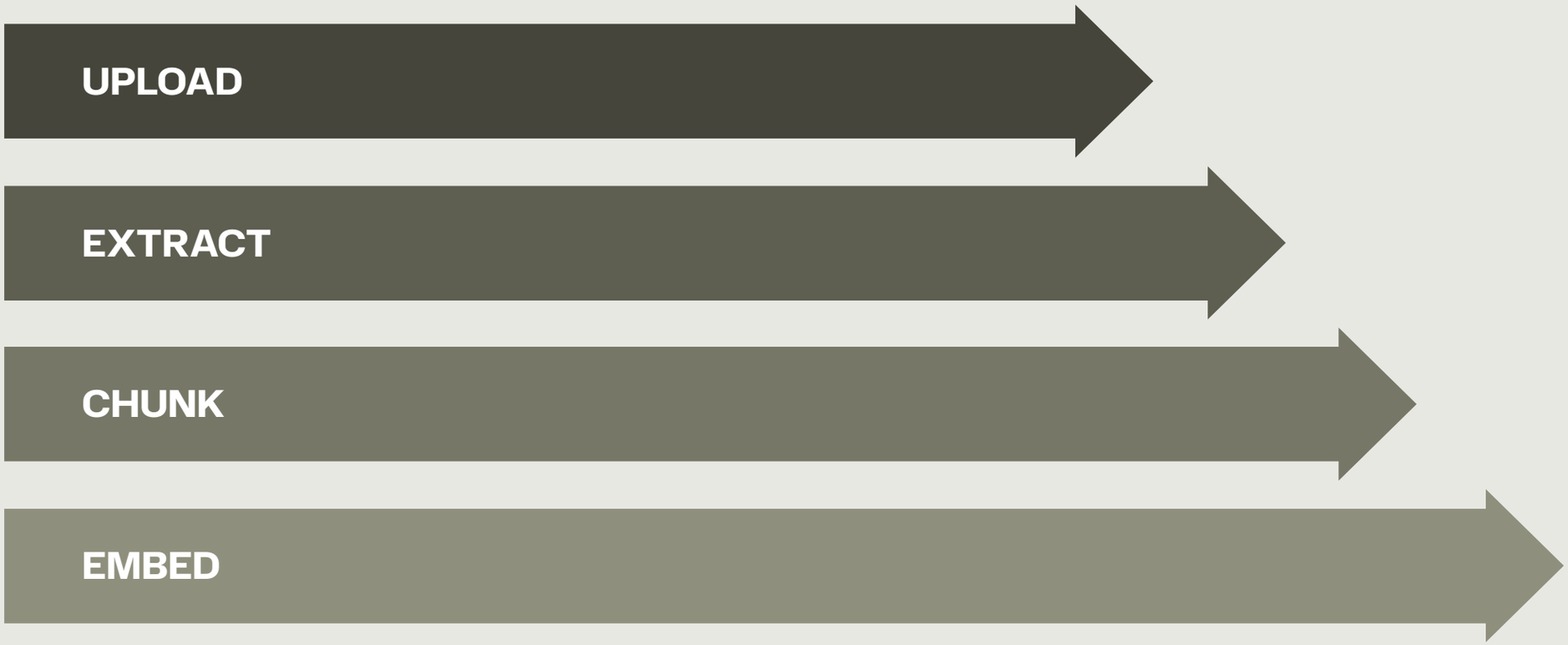
SEVEN LAYERS, TWO DATA PATHS

Ingestion runs async and durable. Retrieval runs sync and latency-critical. Separating them keeps slow writes from affecting query speed.

Layer	Components	Role
Frontend	Next.js · Zustand · TanStack Query	Chat, upload, citations
API Gateway	FastAPI · CORS · tenant extractor	Routing · middleware · isolation
Ingestion	PDF parser · chunker · embeddings · dual write	Async write path
Retrieval	Embeddings · ChromaDB · threshold filter	Sync query path
Generation	GPT-4.1-mini · prompt builder · citation repair	Grounded answer
Observability	`query_logs` · structured logs · `/inspect`	Audit · latency · quality
Infrastructure	Docker Compose · PG 15 · ChromaDB · volumes	Deploy · persist

The two paths share only the vector store and metadata DB. Ingestion returns `202` immediately; query stays fully synchronous and instrumented.

INGESTION PIPELINE: ASYNC DOCUMENT PROCESSING



Documents are accepted immediately, then processed in the background. The pipeline is simple: **Upload → Extract → Chunk → Embed**. A job reference can be returned right away, while status is checked separately.

RETRIEVAL PIPELINE: SEMANTIC SEARCH

No prompt engineering compensates for bad retrieval. If the wrong chunks reach the LLM, the answer will be wrong or hallucinated. The retrieval pipeline is the quality-critical path.

1

EMBED QUERY

Encode the user question with the same embedding model used at ingestion.

2

CHROMA SEARCH

Run cosine similarity search over the tenant namespace and return top-k candidates.

3

THRESHOLD FILTER

Final tuned cutoff: score ≥ 0.55. We started at 0.65, then lowered it after evaluation improved retrieval coverage without materially increasing hallucination risk.

4

ASSEMBLE CONTEXT

Sort chunks by score and label them `[Src N]` for citation rendering.

DEBUGGING

`/query/inspect` is the retrieval-only endpoint for diagnosing quality without GPT cost.

OBSERVED METRICS

- **Score threshold:** 0.55
- **Chroma search:** 13ms
- **Retrieval total:** 24ms
- **Chunks returned:** 5

GENERATION: HALLUCINATION DEFENSE

The generation step uses three controls to reduce hallucinations: a strict system instruction, a context assembly step that only passes validated sources, and a citation repair pass when the model omits source markers.

L1 — SYSTEM INSTRUCTION

Answer only from provided context. If the context is not enough, say so.

L2 — CONTEXT ASSEMBLY

Only scored chunks are included. They are labeled [Src N] before reaching the model.

L3 — CITATION REPAIR

If the response is missing citations, run a repair pass instead of serving it as-is.

CITATION REPAIR LOOP

If GPT-4.1-mini returns an answer without inline [Src N] markers, the system detects it and retries. The goal is to avoid uncited answers, not to trust the first pass.

BEFORE REPAIR

Citation coverage: 0.0

No citations in first pass. Response text generated but no [Src N] markers present.

AFTER REPAIR

Citation coverage: 0.917

91.7% of claims cited. 5 distinct sources referenced. One additional LLM call. Latency cost: ~4–6s.

OBSERVABILITY ARCHITECTURE

Each request gets a UUID at middleware and carries `request_id` and `tenant_id` through the full stack. Logs are structured, searchable, and isolated by tenant.

STRUCTURED LOG EVENTS

- `request_completed` – method, path, status_code, duration_ms, tenant_id
- `llm_call_completed` – model, tokens, total_ms
- `citation_repair_triggered` – request_id, original_coverage
- `ask_completed` – answered, citations_count, total_ms

TENANT-AWARE QUERY LOG

```
CREATE TABLE query_logs (
  id           UUID PRIMARY KEY,
  tenant_id    VARCHAR,
  request_id   UUID,
  query_text   TEXT,
  response_text TEXT,
  latency_ms   INT,
  was_answered  BOOLEAN
);
```

REQUEST ID THREADING

One ID from ingress to downstream services for traceability.

STRUCTURED LOGGING

Consistent event names and fields for debugging and dashboards.

TENANT ISOLATION

Every record is keyed by tenant_id for filtering, review, and access control.

FRONTEND

FRONTEND ARCHITECTURE

State responsibilities are split cleanly: Zustand handles UI state, and TanStack Query handles server state.

CHAT UI

Message list render, input handling, latency footer (`5 sources · 13625ms`). Multi-turn grounded chat with decreasing latency per turn reflecting cache warmup.

CITATION RENDERER

Parses `[Src N]` markers → numbered badges with filename/page tooltip. Footer deduplication pass over all source references assembled at render time.

UPLOAD INTERFACE

PDF drag-and-drop → multipart POST → TanStack polling until `status:ready`. Exponential backoff on status poller.

RETRIEVAL INSPECTOR

Dev panel calling `/query/inspect`. Renders chunk scores, threshold, and per-stage latency — no LLM cost.

DEBUGGING

PRODUCTION BUGS ENCOUNTERED

Bug	Root Cause	Lesson
Wrong tenant — no results, no error	Docs went to `demo-tenant`; queries hit `default`.	Check tenancy first. Silent empties are still bugs.
Every query 404'd	Frontend called `/api/v1/query`; backend served `/api/v1/query/ask`.	Lock the API contract down early.
Status poll 500	`document_chunks_count` vs `chunk_count`.	Validate response models with real serialization paths.
Chroma in git	Local SQLite state was committed in `./chroma-data/`.	Keep runtime data out of the repo.

Key pattern: /inspect exposed retrieval issues fast, without paying LLM costs. Diagnostics saved the day.

SYSTEM PERFORMANCE

370MS
UPLOAD HANDOFF

24MS
QUERY EMBED

18MS
CHROMA SEARCH

42MS
RETRIEVAL TOTAL

4.5S
GENERATION LATENCY

P0 — BEFORE REAL TRAFFIC

- **Celery + Redis** — durable, retryable ingestion queue
- **SSE streaming** — reduce time to first token
- **JWT auth** — tenant from verified token, not client header
- **Transactional dual-write** — saga pattern or pgvector migration

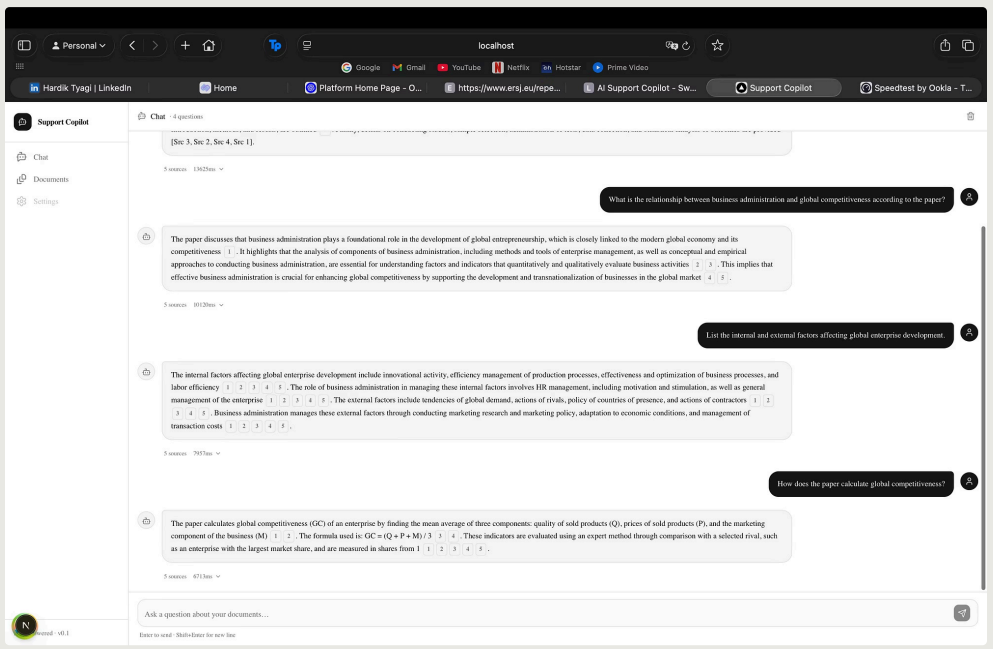
QUALITY & SCALE

- **Cross-encoder re-ranking** — Cohere Rerank between retrieval and generation
- **Per-tenant threshold config** — tune via labeled eval set
- **Hybrid retrieval** — BM25 + semantic via Reciprocal Rank Fusion
- **HyDE + query expansion** — improve sparse-query recall

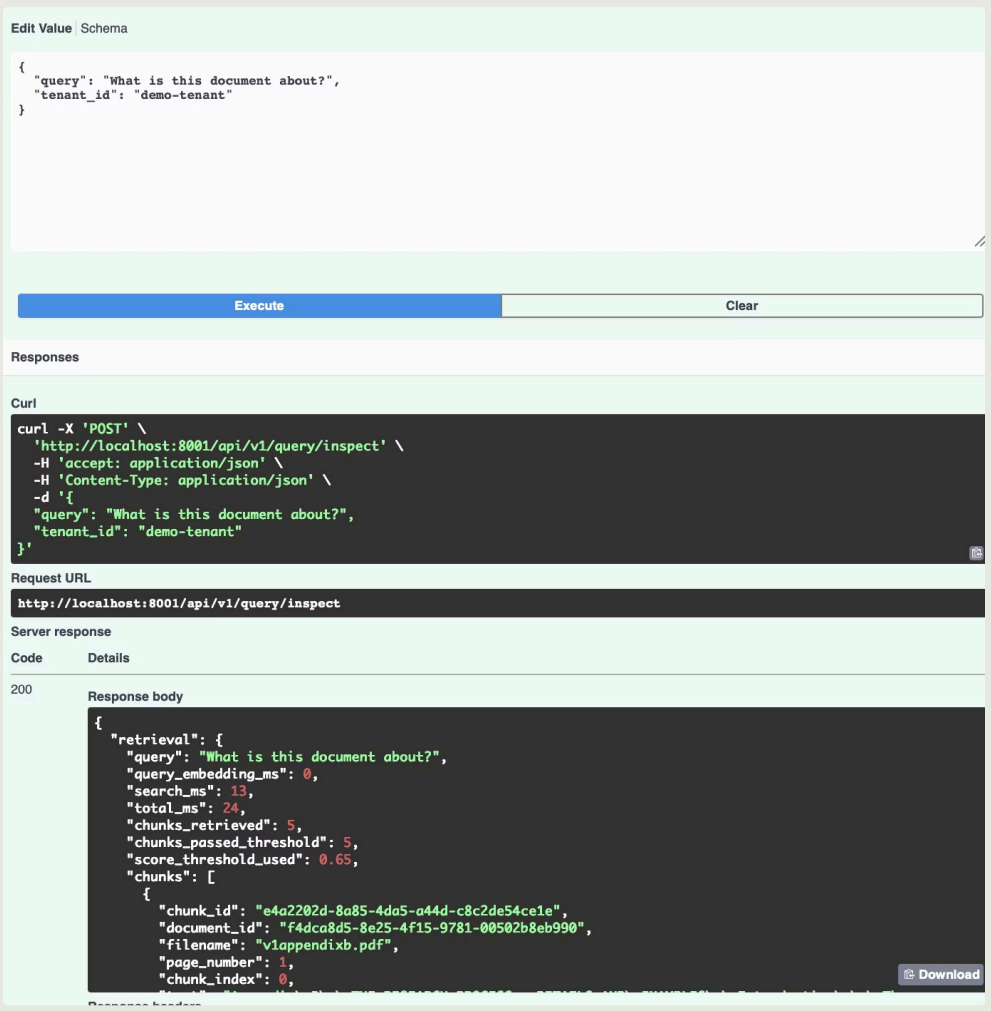
Key learnings: Retrieval quality is the ceiling — optimize it before touching the prompt. Embedding model choice is a commitment — add `model_version` from day one. Structured logging with request IDs makes every debug session trivial. Citation coverage is a proxy for hallucination rate — track it per tenant over time.

ENGINEERING EVIDENCE: PRODUCTION VALIDATION

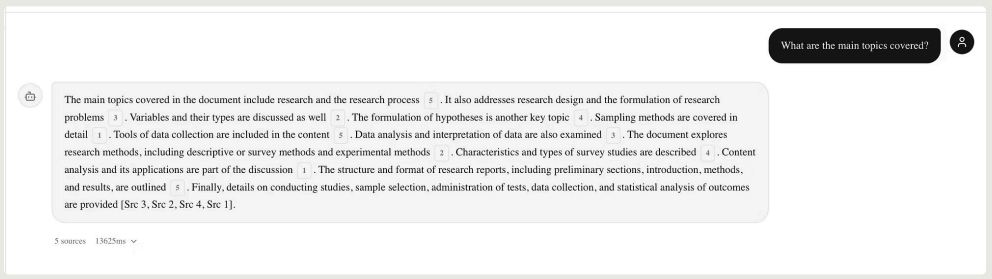
Real screenshots from the running system



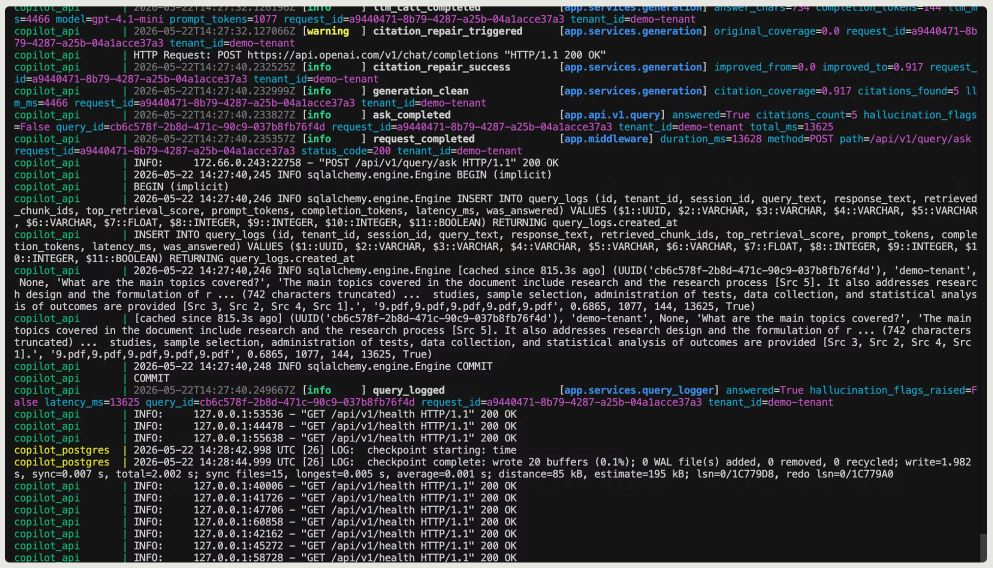
Chat UI with Citation Grounding — Multi-turn conversation with inline [Src N] markers and source attribution.



Retrieval Inspector (/query/inspect) — Zero-cost debugging endpoint showing chunk scores and threshold filtering.



Citation Repair in Action — 91.7% citation coverage after repair loop.



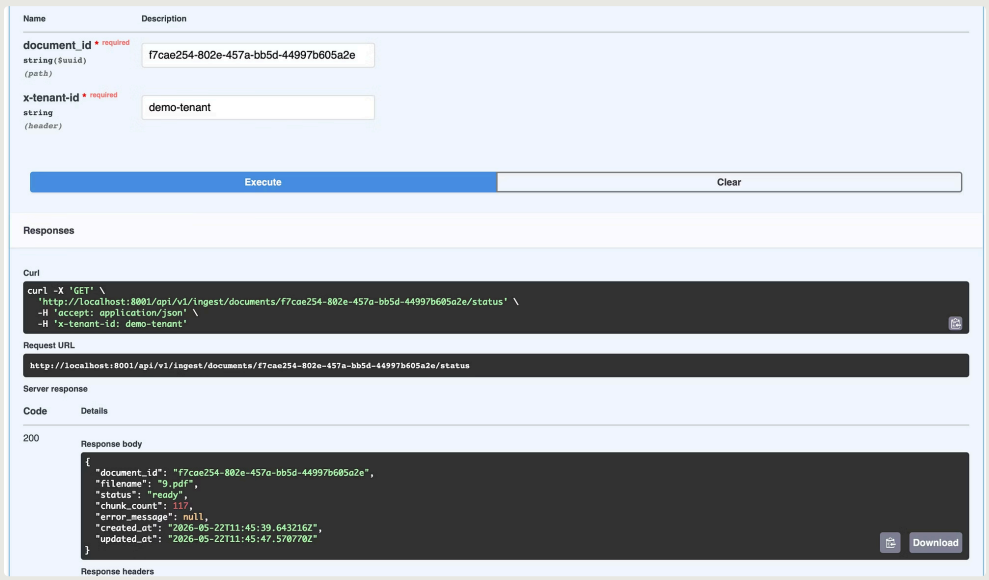
Structured Query Logs — Request ID threading, tenant isolation, per-stage latency tracking.

All screenshots captured from production-grade implementation. Request IDs, tenant isolation, and latency metrics are real measured values.

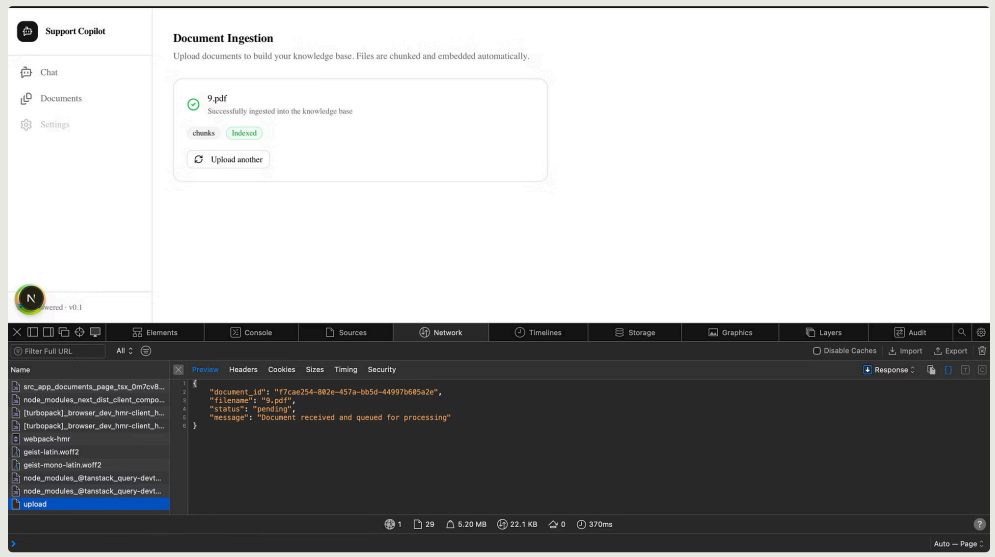
INFRASTRUCTURE

ENGINEERING EVIDENCE: INFRASTRUCTURE & INGESTION

Async processing and upload validation



Async Ingestion Status — 202 Accepted pattern with job reference and polling endpoint.



Upload Interface — Drag-and-drop PDF ingestion with real-time status polling.

KEY IMPLEMENTATION DETAILS

- **202 Accepted Pattern** — HTTP handler returns immediately with job reference
- **Async Processing** — Background worker processes PDF extraction, chunking, embedding
- **Status Polling** — Frontend polls /status endpoint with exponential backoff
- **Dual Write** — Chunks written to both PostgreSQL and ChromaDB for durability